ECE

Major: Cybersecurity International

Course
Sécurité des Containers

Lab Report
Session 2

Instructor
Etienne LOUTSCH

Group Members

- ARDILLON Bastian
- SAVEAUX Octave
- BRULEY Martin

Date
February 25, 2026

Session 2 : Bonnes Pratiques de Sécurité

Gestion des Réseaux et des Secrets

Sécurisation Réseau

1. Utilisation de réseaux Docker isolés :

```
docker network create --driver bridge secure-net
```

```
arbasti@arbasti-Desktop:~$ docker network create --driver bridge secure-net
201f8fc9bd6bd0ece16773e637364171e05c066ced696e23f49f19e310c5c404
arbasti@arbasti-Desktop:~$ docker network ls
NETWORK ID     NAME         DRIVER    SCOPE
dcc9991ed101   bridge       bridge    local
1e3665d4ed4f   host         host      local
efa8cd8e1ad4   none         null      local
201f8fc9bd6b   secure-net   bridge    local
arbasti@arbasti-Desktop:~$ docker network inspect secure-net
[
    {
        "Name": "secure-net",
        "Id": "201f8fc9bd6bd0ece16773e637364171e05c066ced696e23f49f19e310c5c404",
        "Created": "2026-02-18T13:21:13.52292853Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false
```

Figure 0: Création d'un réseau Docker bridge isolé

La création d'un réseau Docker dédié permet d'isoler les communications entre containers. Ceci va limite l'exposition au réseau externe et réduire la surface d'attaque

Gestion des Secrets

1. Éviter l'Exposition Involontaire de Ports

Nous lançons un container avec un port exposé :

```
docker run -d -p 8080:80 nginx
```

And we check if the port is exposed using `netstat` or `ss` :

```
arbasti@arbasti-Desktop:~$ docker run -d -p 8080:80 nginx
f74c4ab7a48594dca22c5b5670c2b01d3f5986e79522d2aa5c3367ff7ea7c177
arbasti@arbasti-Desktop:~$ ss -tulnp | grep 8080
tcp   LISTEN 0      4096               *:8080           *:*
```

Figure 1: Vérification des ports exposés<

L'exposition de ports rend un service accessible depuis l'extérieur. Les ports doivent être contrôlée pour éviter des accès non autorisés

---

2. Restreindre les permissions d'accès aux fichiers sensibles

Nous montons un fichier sensible en lecture seule :

```
docker run -it --rm -v /etc/passwd:/mnt/passwd:ro alpine sh
```

```
arbasti@arbasti-Desktop:~$ docker run -it --rm -v /etc/passwd:/mnt/passwd:ro alpine sh
/ # cat /mnt/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
```

Figure 2: Accès en lecture seule à un fichier sensible dans le container

Maintenant essayons de modifier le fichier

```
/ # echo test >> /mnt/passwd
sh: can't create /mnt/passwd: Read-only file system
/ #
```

Figure 3: Écriture refusée sur un fichier monté en lecture seule

L'utilisation du mode lecture seule empêche toute modification du fichier `/mnt/passwd`

---

3. Auditer la configuration d'un container avec Docker Bench

Nous installons et exécutons Docker Bench for Security :

```
git clone https://github.com/docker/docker-bench-security.git
cd docker-bench-security/
```

L'audit sur l'hôte va nous donner un score de securité basé sur des vérifications (passed/failed)

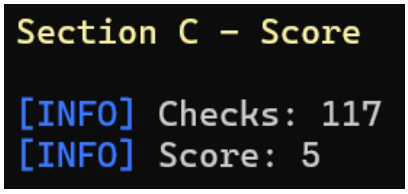

Figure 4: Résultats de l'audit de sécurité sur l'hôte

Nous auditons également un container vulnérable (`vulnerables/web-dvwa`) :

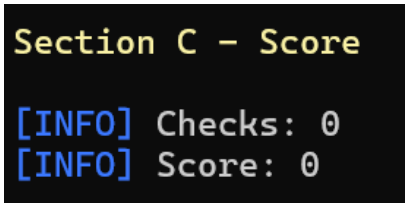

Figure 5: Résultats d'audit sur un container volontairement vulnérable

La majorité des vérifications échouent car le container est volontairement mal sécurisé, nous affichant des erreurs classiques (ports exposés, permissions faibles, logiciels obsolètes).

---

4. Stocker et Utiliser des Secrets

- Lancer un container Vault :

Nous lançons un container Vault en server mode avec la command suivante :

```
docker run --cap-add=IPC_LOCK -e 'VAULT_LOCAL_CONFIG={"storage": {"file": {"path": "/vault/file"}}, "listener": [{"tcp": { "address": "0.0.0.0:8200", "tls_disable": true}}], "default_lease_ttl": "168h", "max_lease_ttl": "720h", "ui": true}' -p 8200:8200 hashicorp/vault server
```

```
arbasti@arbasti-Desktop:~$ docker run --cap-add=IPC_LOCK -e 'VAULT_LOCAL_CONFIG={"storage": {"file": {"path": "/vault/file"}}, "listener": [{"tcp": { "address": "0.0.0.0:8200", "t
ls_disable": true}}], "default_lease_ttl": "168h", "max_lease_ttl": "720h", "ui": true}' -p 8200:8200 hashicorp/vault server
==> Vault server configuration:

Administrative Namespace:
                     Cgo: disabled
   Environment Variables: GOTRACEBACK, HOME, HOSTNAME, NAME, PATH, PWD, SHLVL, VAULT_LOCAL_CONFIG
             Go Version: go1.25.6
             Listener 1: tcp (addr: "0.0.0.0:8200", cluster address: "0.0.0.0:8201", disable_request_limiter: "false", max_json_array_element_count: "10000", max_json_depth: "300
", max_json_object_entry_count: "10000", max_json_string_value_length: "1048576", max_request_duration: "1m30s", max_request_size: "33554432", tls: "disabled")
               Log Level:
                   Mlock: supported: true, enabled: true
           Recovery Mode: false
                 Storage: file
                 Version: Vault v1.21.3, built 2026-02-03T14:56:30Z
             Version Sha: f4f0f4eb7f467bbc99ec89121e1d1ad9c3d78558

2026-02-25T13:49:38.655Z [INFO]  proxy environment: http_proxy="" https_proxy="" no_proxy=""
2026-02-25T13:49:38.655Z [INFO]  incrementing seal generation: generation=1
2026-02-25T13:49:38.655Z [WARN]  no `api_addr` value specified in config or in VAULT_API_ADDR; falling back to detection if possible, but this value should be manually set
2026-02-25T13:49:38.780Z [INFO]  core: Initializing version history cache for core
2026-02-25T13:49:38.780Z [INFO]  events: Starting event system
==> Vault server started! Log data will stream in below:

2026-02-25T13:49:45.920Z [INFO]  core: security barrier not initialized
2026-02-25T13:49:45.920Z [INFO]  core: seal configuration missing, not initialized
2026-02-25T13:49:45.921Z [INFO]  core: security barrier not initialized
```

Figure 6: Lancement du serveur Vault dans un container

La commande utilisé lance un serveur Vault avec un stockage local et une interface web activée. L'option `IPC_LOCK` empêche l'écriture des données sensibles sur disque

Nous vérifions que le container tourne correctement :

```
docker ps
```

```
arbasti@arbasti-Desktop:~$ docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED         STATUS         PORTS                                       NAMES
5c1691cecc59   hashicorp/vault   "docker-entrypoint.s…"   4 minutes ago   Up 4 minutes   0.0.0.0:8200->8200/tcp, [::]:8200->8200/tcp   distracted_lalande
```

Figure 7: Vérification que le container Vault est en cours d'exécution

Passons à l'initialisation et dévérouillage du Vault :

```
docker exec -it distracted_lalande vault operator init -key-shares=1 -key-threshold=1
```

On rajoute les options :

- -key-shares=1 : ceci générera une seule clé
- -key-threshold=1 : nous aurons besoin que d'une clé pour dévérouiller le Vault à l'avenir

```
arbasti@arbasti-Desktop:~$ docker exec -it distracted_lalande vault operator init -key-shares=1 -key-threshold=1
WARNING! VAULT_ADDR and -address unset. Defaulting to https://127.0.0.1:8200.
Get "https://127.0.0.1:8200/v1/sys/seal-status": http: server gave HTTP response to HTTPS client
arbasti@arbasti-Desktop:~$ docker exec -it distracted_lalande sh -c 'export VAULT_ADDR=http://127.0.0.1:8200 && vault operator init -key-shares=1 -key-threshold=1'
Unseal Key 1: B+Ds6CcKoEQnAOL9VNiY0qfFaVqB+etipDQduJKFllc=

Initial Root Token: hvs.nt6VSRYJ4PGWPzEDdbBAopEy

Vault initialized with 1 key shares and a key threshold of 1. Please securely
distribute the key shares printed above. When the Vault is re-sealed,
restarted, or stopped, you must supply at least 1 of these keys to unseal it
before it can start servicing requests.

Vault does not store the generated root key. Without at least 1 keys to
reconstruct the root key, Vault will remain permanently sealed!

It is possible to generate new unseal keys, provided you have a quorum of
existing unseal keys shares. See "vault operator rekey" for more information.
```

Figure 8: Initialisation du Vault et génération des clés

`VAULT_ADDR=http://127.0.0.1:8200` permet de définir l'adresse du serveur Vault utilisé par la CLI

```
docker exec -it distracted_lalande sh -c 'export VAULT_ADDR=http://127.0.0.1:8200 && vault operator init -key-shares=1 -key-threshold=1'
```

Déverrouillons le Vault :

```
docker exec -it distracted_lalande sh -c 'export VAULT_ADDR=http://127.0.0.1:8200 && vault operator unseal B+Ds6CcKoEQnAOL9VNiY0qfFaVqB+etipDQduJKFllc='
```

Vue que nous avons rajouté `key-threshold=1` plutôt, nous aurons besoin que d'une clé pour déverrouiller le Vault

```
arbasti@arbasti-Desktop:~$ docker exec -it distracted_lalande sh -c 'export VAULT_ADDR=http://127.0.0.1:8200 && vault operator unseal B+Ds6CcKoEQnAOL9VNiY0qfFaVqB+etipDQduJKFllc=
>
```

Figure 9: Processus de déverrouillage de Vault

Cette étape est obligatoire car Vault démarre en mode scellé pour protéger les secrets

On peut également vérifier les logs du Vault pour confirmer que celui-ci c'est bien déverouillé :

```
2026-02-25T14:43:26.310Z [INFO]  core: vault is unsealed
```

Figure 10: Confirmation dans les logs que Vault est déverrouillé

- Se rendre sur l'UI `localhost:8200`, se connecter avec le root-token puis créer un user/mot de passe (on aurait pu utiliser d'autres méthode d'authentification type approle, tls..)

Maintenant nous pouvons nous connecter sur `http://localhost:8200/` avec le `Initial Root Token` reçu plutôt *(Figure 8)*:

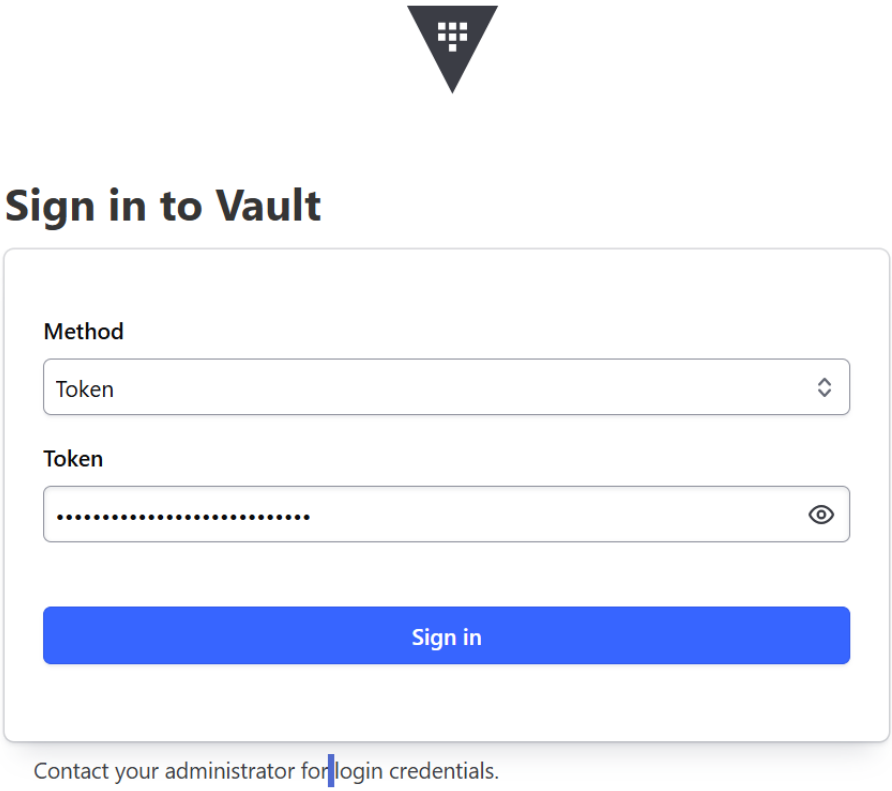

Figure 11: Connexion à Vault avec le token root

Nous sommes maintenant connecté sur `http://localhost:8200/ui/vault/dashboard` :

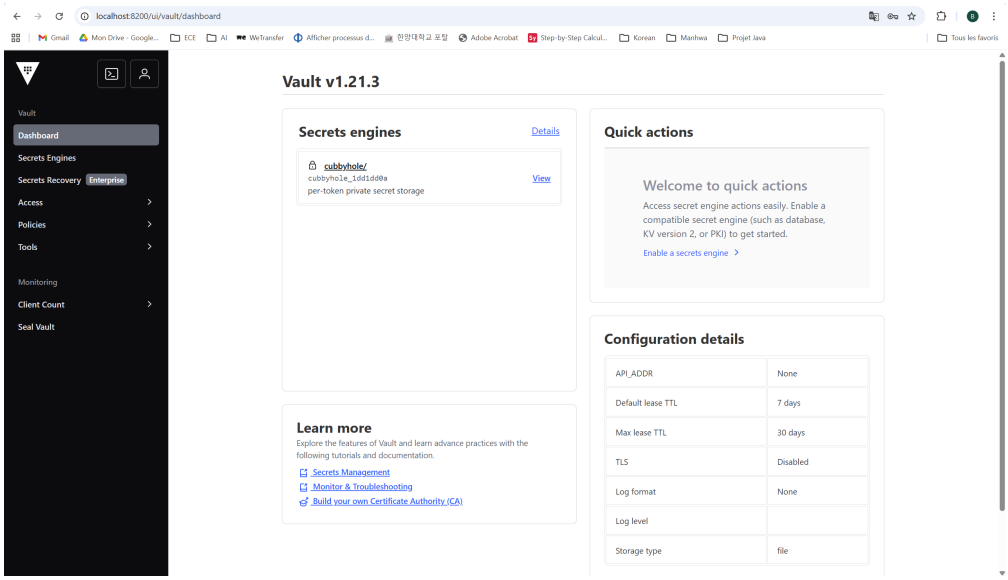

Figure 12: Tableau de bord Vault après authentification

Créons un utilisateur sur l'application :

Figure 13: Création d'un utilisateur avec la méthode userpass

- Créer une ACL qui permet de lire un secret au chemin `containers/mon-secret`

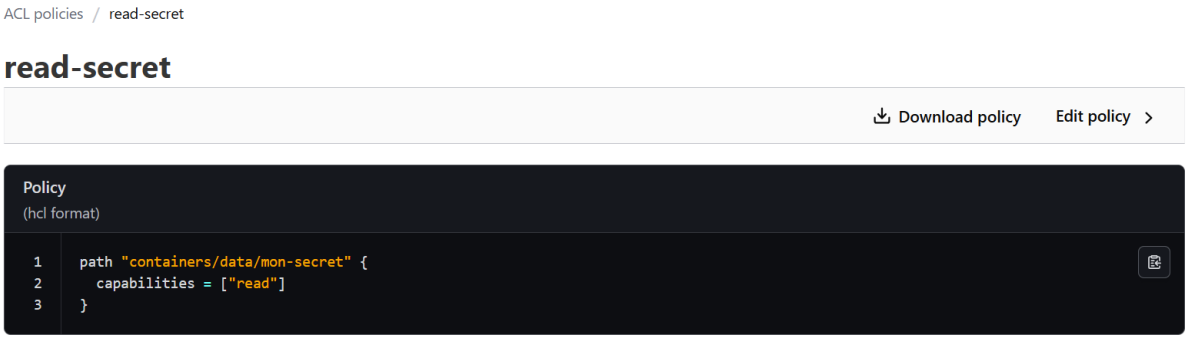

Figure 14: Création d'une politique ACL

Les policies définissent les actions autorisées et sont appliquées lors de la génération de token à la connection de l'utilisateur

- Créer un user/mot de passe et associer l'ACL créer auparavant

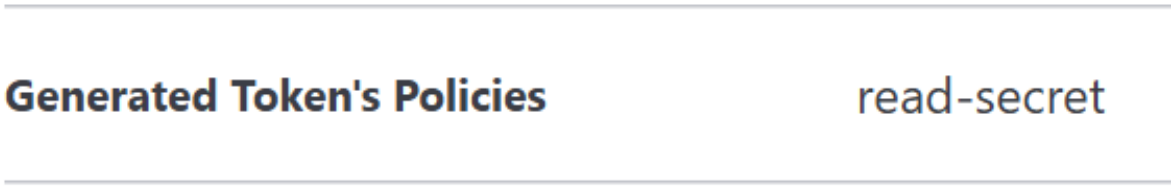

Figure 15: Association d'une policy à un utilisateur

- Créer un secret dans le path `containers/mon-secret`

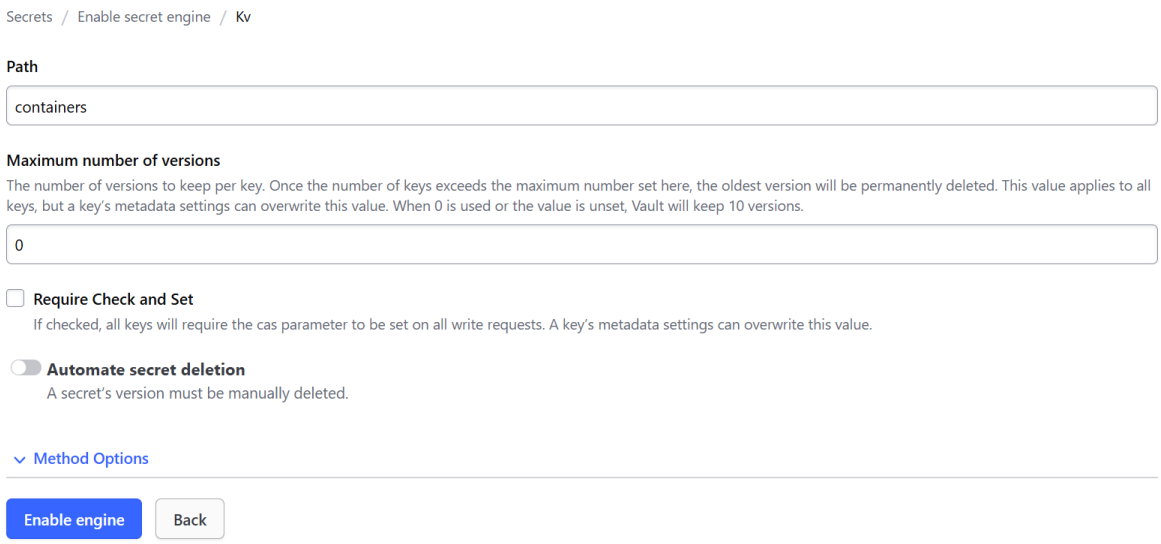

Figure 16: Création du chemin de stockage des secrets

Nous créons un secret `api_key` :

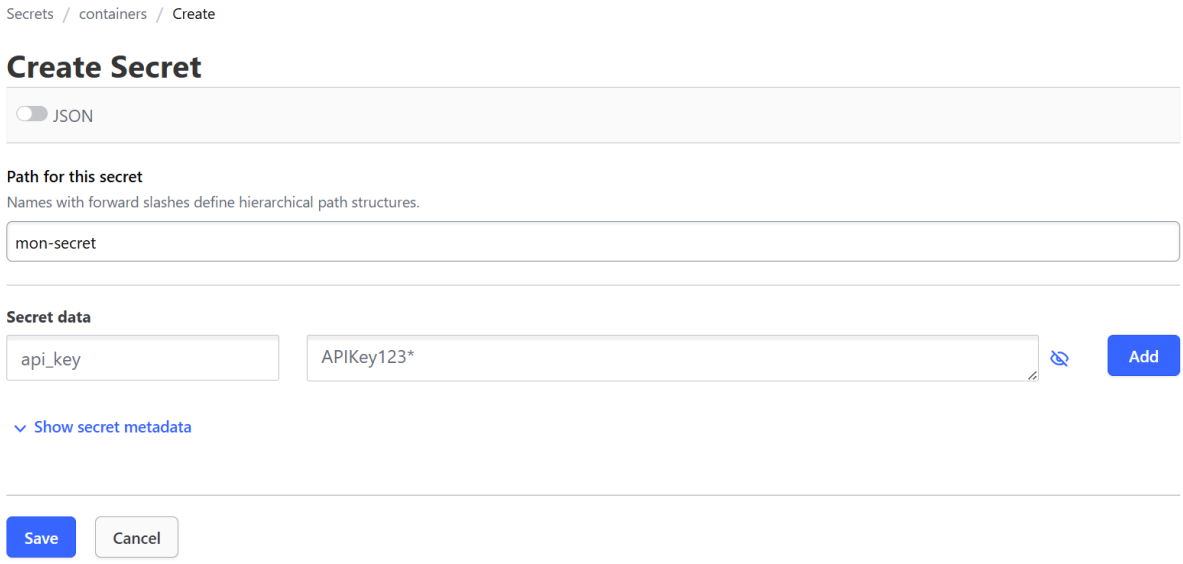

Figure 17: Création d'un secret (clé API)

Voici l'overview de notre nouveau secret :

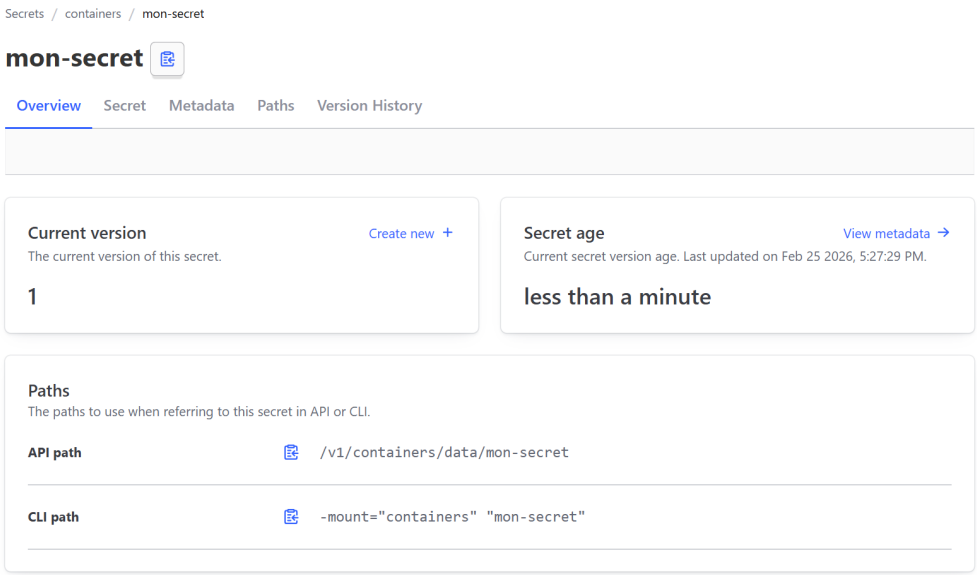

Figure 18: Secret stocké

- Lancer un container alpine et avec l'outil curl faire une authentification au Vault, puis récuperer le secret

```
docker run --rm -it alpine sh
apk add --no-cache curl
```

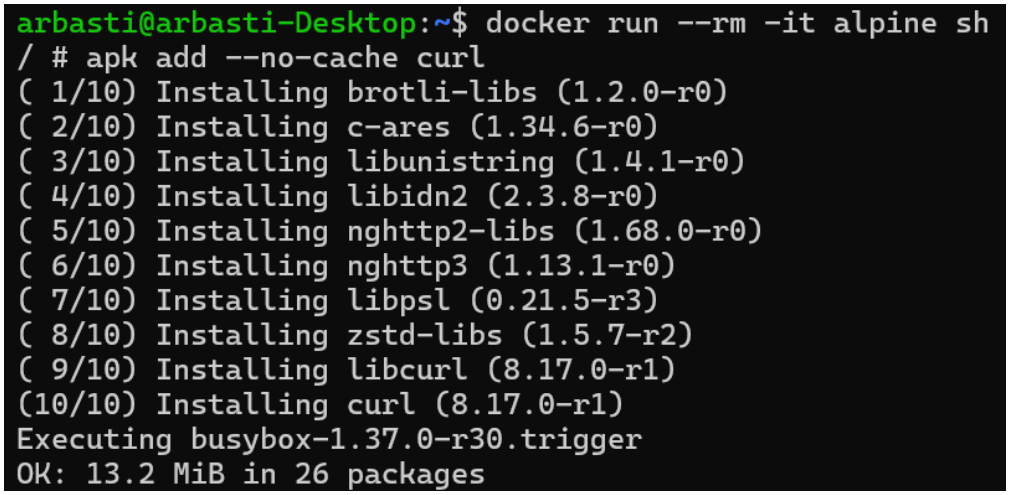

Figure 19: Installation de curl dans un container

Authentification :

```
export VAULT_ADDR=http://host.docker.internal:8200

TOKEN=$(curl -s \
  --request POST \
  --data '{"password":"alpine"}' \
  $VAULT_ADDR/v1/auth/userpass/login/alpine \
  | grep -o '"client_token":"[^"]*' | cut -d':' -f2 | tr -d '"')

echo $TOKEN
```

```
/ # export VAULT_ADDR=http://host.docker.internal:8200
/ #
/ # TOKEN=$(curl -s \
>   --request POST \
>   --data '{"password":"alpine"}' \
>   $VAULT_ADDR/v1/auth/userpass/login/alpine \
>   | grep -o '"client_token":"[^"]*' | cut -d':' -f2 | tr -d '"')
/ #
/ # echo $TOKEN
hvs.CAESIPYsi6xSfNfJaoawGOONJEiRWA4tJSE-3FALBilU4M55Gh4KHGh2cy5tcUZYUURNdGwzZVVzZFpGOU9LUmlOeXo
```

Figure 20: Récupération d'un token d'authentification Vault

Le token permet d'authentifier la requête suivante, qui recupère le secret :

```
curl -s -H "X-Vault-Token: $TOKEN" $VAULT_ADDR/v1/containers/data/mon-secret
```

```
/ # curl -s -H "X-Vault-Token: $TOKEN" $VAULT_ADDR/v1/containers/data/mon-secret
{"request_id":"cf510dd5-4ec1-4421-9671-117c03a789b5","lease_id":"","renewable":false,"lease_duration":0,"data":{"data":{"api_key":"APIKey123*"},"metadata":{"created_time":"2026-02
-25T16:27:29.861424814Z","custom_metadata":null,"deletion_time":"","destroyed":false,"version":1}},"wrap_info":null,"warnings":null,"auth":null,"mount_type":"kv"}
/ #
```

Figure 21: Récupération du secret via l'API Vault

---

5. Trouver la clé Contexte : Un développeur pas très habile (moi) à builder une image docker. Celle-ci comporte une clé API utilisé dans une requête assez suspecte... En tant qu'attaquant, trouver la clé API dans l'image docker

Nous téléchargeons l'image Docker :

```
docker pull ety92/demo:v1
```

```
arbasti@arbasti-Desktop:~$ docker pull ety92/demo:v1
v1: Pulling from ety92/demo
56e759f1f975: Pull complete
Digest: sha256:700771111f9e51b5aeb717a6992fe1de3890203344648bd354f151bde230e032
Status: Downloaded newer image for ety92/demo:v1
docker.io/ety92/demo:v1
```

Figure 22: Téléchargement de l'image Docker

En utilisant la commande suivante, nous pouvons inspecter toutes les couches de l'image Docker ainsi que les instructions utilisées lors de sa construction :

```
docker history ety92/demo:v1
```

```
arbasti@arbasti-Desktop:~$ docker history ety92/demo:v1
IMAGE          CREATED        CREATED BY                                      SIZE      COMMENT
700771111f9e   12 months ago  RUN /bin/sh -c apk add curl &&     curl -H "…   7.51MB    buildkit.dockerfile.v0
<missing>      13 months ago  CMD ["/bin/sh"]                                 0B        buildkit.dockerfile.v0
<missing>      13 months ago  ADD alpine-minirootfs-3.21.3-x86_64.tar.gz /…   8.5MB     buildkit.dockerfile.v0
```

Figure 23: Historique complet de l'image avec les commandes de build

Nous pouvons accéder à l'historique complet de l'image avec la commande suivante :

```
docker history ety92/demo:v1 --no-trunc
```

```
arbasti@arbasti-Desktop:~$ docker history ety92/demo:v1 --no-trunc
IMAGE                                                                     CREATED        CREATED BY
           SIZE      COMMENT
sha256:700771111f9e51b5aeb717a6992fe1de3890203344648bd354f151bde230e032   12 months ago  RUN /bin/sh -c apk add curl &&     curl -H "API-Key: U-never-will-saw-that" -L google.com
 # buildkit   7.51MB    buildkit.dockerfile.v0
<missing>                                                                 13 months ago  CMD ["/bin/sh"]
           0B        buildkit.dockerfile.v0
<missing>                                                                 13 months ago  ADD alpine-minirootfs-3.21.3-x86_64.tar.gz / # buildkit
           8.5MB     buildkit.dockerfile.v0
```

Figure 24: Historique complet de l'image montrant toutes les couches et instructions de build

---

END